

1. In this quiz, we bring together all the concepts covered in this module and outline each of the steps involved in PCA.

1 point

Here is an outline of the steps we will be taking:

- Step 1: Normalize the data
- Step 2: Construct the covariance matrix of the data
- Step 3: Calculate the eigenvector and eigenvalues of the covariance matrix. We call the eigenvector with the largest eigenvalue the principal component eigenvector
- Step 4: Find the new data of reduced dimensionality

Before we go through each of the steps individually, we will first consider how to fit a line to some data. Let's consider two different ways of calculating a line of best fit.

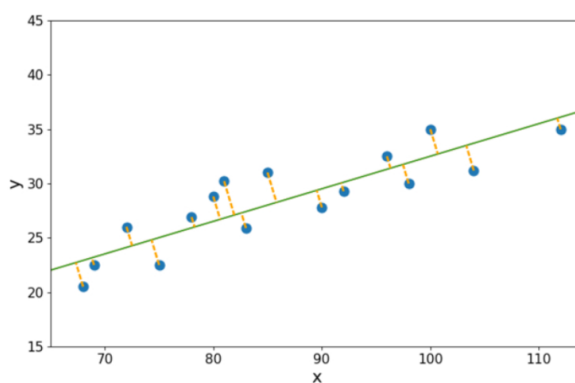
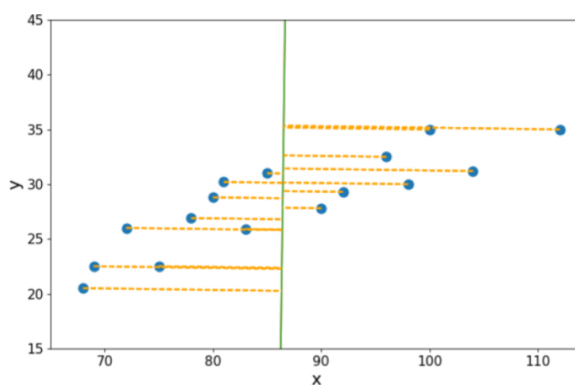
The line of best fit is either

1. a line which maximises the spread of the data points when projected onto this line
- or
2. a line which minimises the sum of squared distances between the original data points and the projected data points.

In fact, these two descriptions have exactly the same results.

To demonstrate this, look at the following two figures which are plots of y as a function of x . y and x represent two different variables. The above plot shows a "bad" fit since the spread of the data points when projected onto the fitted line is much less than that of the bottom plot which shows a "good" fit. At the same time, the sum of squared distances of the "bad" fit is much greater than that of the "good" fit.

The orange dotted lines in the plot below show the orthogonal projection of each of the data points onto the fitted line.



Test this out for yourself using the code cell below by changing the value for "m" which represents the gradient of the line of best fit for the data, and then run the cell.

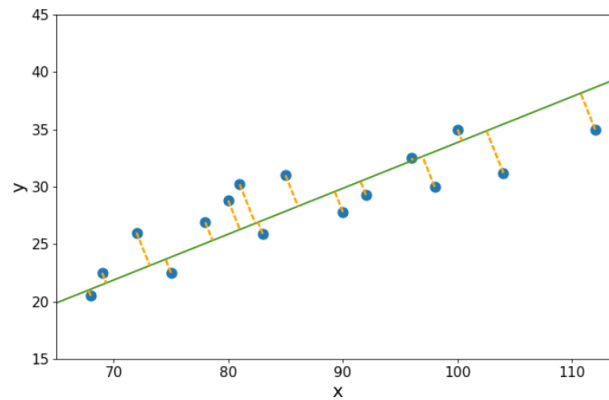
```
1 plt.plot(x, y, 'o', markersize=10)
2 m = 0.4 #Change this value
3 plot_line(m)
```

Run

Reset

Sum of squared distances: 87.5123922414

Spread of data: 40.481583722597314



What value of m (gradient) gives a sum of squared distance value of less than 100 (give your answer to 2dp)?

0.4

2. Before we start the first step of PCA, let's also recall some important definitions of some of the mathematical concepts we will be using throughout the process.

1 point

A measure of the correlation between different variables is given by the:

- ☐ Projection matrix
☒ Covariance matrix
☐ Eigenvalues

3. The directions in which data is spread out is given by the:

1 point

- ☒ Eigenvectors
☐ Covariance matrix
☐ Eigenvalues

4. The magnitude of the spread of data along the principal components is given by the:

1 point

- ☐ Determinant of the covariance matrix
☐ Inner product
☒ Eigenvalues

5. Now you will normalize the data from question 1.

1 point

Normalizing data generally involves two steps:

1. subtracting the mean from the data
2. dividing the data by the standard deviation

In some implementations, both steps are essential, however in this case we will ignore this second step for reasons beyond the scope of this quiz.

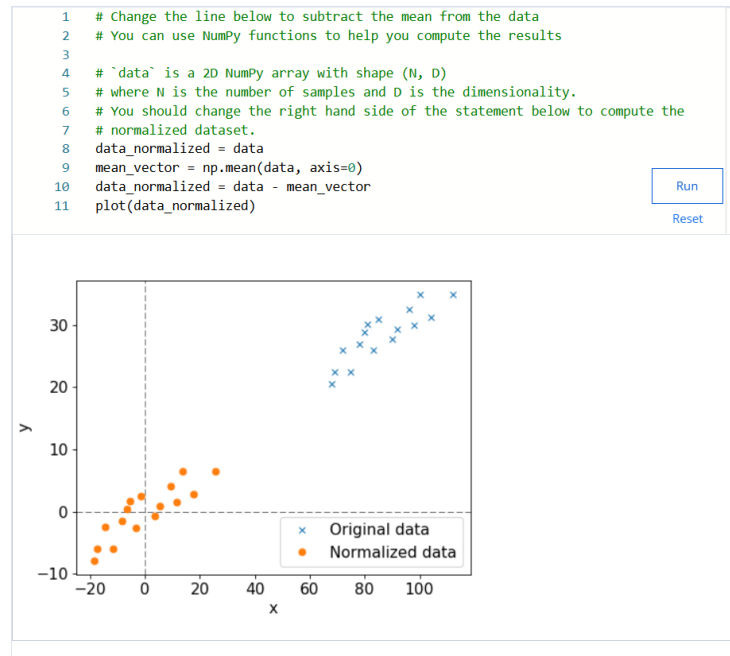
However, it is still important to center the data by subtracting the mean. If this initial step is missed, the 1st principal component may not fit the data along the main direction, and will not be reflective of the correlations and spread in the data.

The data for the plots is stored in the 2D array `data`, with the first column `data[:, 0]` representing the first variables x , and the second column `data[:, 1]` representing the second variables y . You will calculate a new array `data_normalized`, which contains the data minus the mean of each variable.

Edit the code below to normalize (center) the data, and run the cell to check if the plot produced is what you expect. You may wish to use `np.mean`; see <https://numpy.org/doc/stable/reference/generated/numpy.mean.html> .

The plot shows the normalized data when it is calculated. Check that you get what you would expect before completing the rest of the exercise.

The dotted lines shown in the figure produced by the below code block mark the x and y axis.



When you are happy that your code calculates `data_normalized` correctly, copy the code for the calculation into the cell below and run the cell to submit your answer.

```

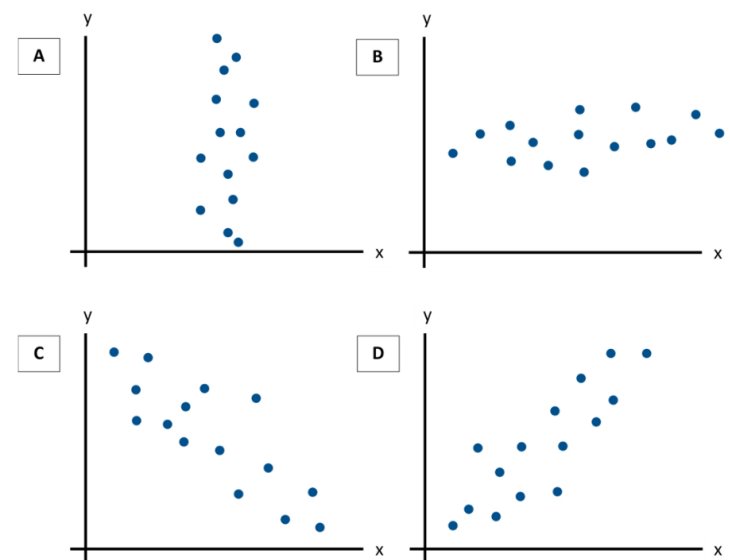
1 # Copy your code from above to calculate data_normalized correctly
2 data_normalized = data
3 mean_vector = np.mean(data, axis=0)
4 data_normalized = data - mean_vector
5 plot(data_normalized)

```

6. Before we move onto the next step where we construct the covariance matrix, let's recall the main features of a covariance matrix.

1 point

Match the plots with the corresponding covariance matrix - note that you do not need to calculate anything; use what you know about features of a covariance matrix and what it represents to match the following plots to the corresponding covariance matrix.



- ☐ $A = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}, B = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 3 \end{bmatrix}, C = \begin{bmatrix} 6 & 0 \\ 0 & 3 \end{bmatrix}, D = \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}$
☐ $A = \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}, B = \begin{bmatrix} 6 & 0 \\ 0 & 3 \end{bmatrix}, C = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 3 \end{bmatrix}, D = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}$
☒ $A = \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}, B = \begin{bmatrix} 6 & 0 \\ 0 & 3 \end{bmatrix}, C = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}, D = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 3 \end{bmatrix}$

- ☐ $A = \begin{bmatrix} 6 & 0 \\ 0 & 3 \end{bmatrix}, B = \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}, C = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}, D = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 3 \end{bmatrix}$
☐ $A = \begin{bmatrix} 6 & 0 \\ 0 & 3 \end{bmatrix}, B = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 3 \end{bmatrix}, C = \begin{bmatrix} 2 & 0 \\ 0 & 7 \end{bmatrix}, D = \begin{bmatrix} 4 & -2 \\ -2 & 5 \end{bmatrix}$

$$\begin{bmatrix} \mu_x & \sigma_x \\ \mu_y & \sigma_y \end{bmatrix} \quad \begin{bmatrix} \mu_x & \sigma_x \\ \mu_y & \sigma_y \end{bmatrix} \quad \begin{bmatrix} \mu_x & \sigma_x \\ \mu_y & \sigma_y \end{bmatrix} \quad \begin{bmatrix} \mu_x & \sigma_x \\ \mu_y & \sigma_y \end{bmatrix}$$

7. Now let's move onto the next step, where you will construct the covariance matrix for our data. Recall the following formula for an element of the covariance matrix:

1 point

$$\text{cov}[x, y] = E[(x - \mu_x)(y - \mu_y)] \approx \frac{1}{N} \sum_{i=1}^N (x_i - \mu_x)(y_i - \mu_y)$$

Here, the subscript i refers to the i th entry of a given variable. μ_x represents the mean of x and μ_y represents the mean of y . N represents the number of data points which for our data set is 16.

The covariance matrix itself is given by:

$$A = \begin{bmatrix} \text{cov}[x, x] & \text{cov}[x, y] \\ \text{cov}[y, x] & \text{cov}[y, y] \end{bmatrix}.$$

You will calculate the covariance matrix for the data in the previous questions in the code cell below. Remember that `data_normalized[i, 0]` refers to the normalized i th element in x , given by $(x_i - \mu_x)$. Similarly, `data_normalized[i, 1]` refers to the normalized i th element in y , given by $(y_i - \mu_y)$. Hence `data_normalized[:, 0]` is a 2D array (column vector) containing all the normalized data for variable x , and similarly `data_normalized[:, 1]` is a 2D array (column vector) containing all the normalized data for variable y . The variable `cov_xx` is equivalent to `cov[x, x]`.

Complete the code cell by calculating the remaining elements of the covariance matrix. `cov_xy` has been provided as an example. When you are done, directly copy & paste the output of the code cell into the answer box below.

```
1 # Complete the code below to compute cov[x, x] cov[y, x] and cov[y, y].
2 # cov_xy has been provided as an example
3 cov_xx = (1/N)*np.dot(data_normalized[:, 0], data_normalized[:, 0])
4 cov_xy = (1/N)*np.dot(data_normalized[:, 0], data_normalized[:, 1])
5 cov_yx = (1/N)*np.dot(data_normalized[:, 1], data_normalized[:, 0])
6 cov_yy = (1/N)*np.dot(data_normalized[:, 1], data_normalized[:, 1])
7
8 # This prints the final covariance matrix
9 # using the elements computed above
10 # Once you have finished writing the code above,
11 # DIRECTLY copy the output of the following function
12 # to the answer cell.
13 print_covariance_matrix()
```

Run

Reset

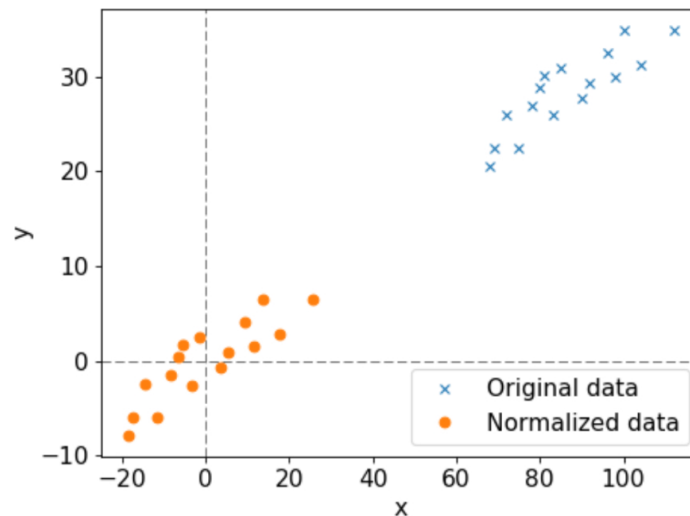
```
[[ 159.62109375  45.23710938]
 [ 45.23710938  16.99496094]]
```

```
[[ 159.62109375 45.23710938] [ 45.23710938 16.99496094]]
```

8. In the next step, we will be calculating the eigenvectors and eigenvalues of the covariance matrix that we found in the previous question.

1 point

Here is a plot of the normalized data which shows the correlation between x and y



Using this, in which direction do you think the first principal component eigenvector will point?

- ☒ Upwards and towards the right
- ☐ Upwards and towards the left
- ☐ Downwards and towards the right

9. Finally, we want to find the new lower dimensional representation of the data. Recall from lectures that the coordinates ($\mathbf{X}$) of the projection from 2D to 1D with respect to the orthonormal basis of the principal subspace is given by:

$$\mathbf{X} = \mathbf{B}^T \mathbf{x}_*,$$

where $\mathbf{B}$ is the basis vectors arranged as the columns of a matrix, and $\mathbf{x}_*$ is vector containing the normalized original data. In this case $\mathbf{B}$ is just the first principal eigenvector.

To project the reduced dimension coordinates back into the original 2D space, we use

$$\mathbf{B}\mathbf{B}^T \mathbf{x}_*$$

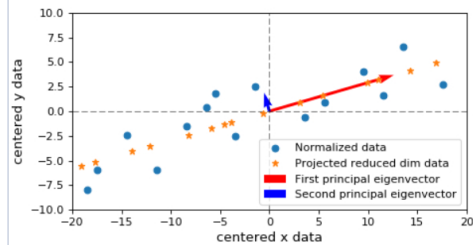
This final step has been done for you, where we combine everything we have found in the previous steps to plot the final projected coordinates.

Run the code cell below (you do not need to edit the code) to convince yourself that we have indeed reduced the dimensionality of the original data set from two to one. Note that the length of the eigenvectors are the square root of the corresponding eigenvalue.

```
1 plot_projected_coords()
```

Run

Reset



Congratulations, you have performed PCA for a small 2d dataset! In this week's assessment you will write PCA code more formally and apply it to higher dimensional datasets!

To wrap-up, which of the following do you think are real-world applications of PCA?

- ☐ PCA can capture non-linear relationships between variables
- ☒ PCA makes it easy to spot patterns and trends in data
- ☒ PCA can be used for image compression and facial recognition
- ☒ PCA saves memory on devices when storing data